# Jest Unit Testing Practice

## Environment setup

```
mkdir university-testing-lab
cd university-testing-lab
npm init -y
npm install --save-dev jest
```

Add the following fields to package.json while retaining installed dependencies:

```
"type": "module",
"scripts": {
  "test": "node --experimental-vm-modules ./node_modules/jest/bin/jest.js --runInBand",
  "test:coverage": "node --experimental-vm-modules ./node_modules/jest/bin/jest.js --coverage --runInBand"
}
```

Create jest.config.js:

```
export default {
  testEnvironment: "node",
  transform: {},
  coverageProvider: "v8",
  collectCoverageFrom: ["src/**/*.js"],
  coverageReporters: ["text", "html", "lcov"]
};
```

```
university-testing-lab/
  src/  registration.js  grading.js  tuition.js
        enrollment.js  scholarship.js
  tests/  unit/  lifecycle/  data-driven/  coverage/
  tests/fixtures/
  jest.config.js  package.json  reports/
```

## Question 1: Student Registration and Validation (5 marks)

**Scenario and requirements.** The admissions office receives student registration requests from a web form. The starter function accepts incomplete and incorrectly typed values, which can create invalid records or unexpected runtime errors. Your assignment is to establish a clear registration contract, expose defects with automated tests, and correct the implementation without changing valid results.

Starter file: src/registration.js

```
export function registerStudent(student) {
  if (!student.name || student.name.trim() === "") {
    throw new Error("Student name is required");
  }
  if (student.age < 18) {
    throw new Error("Student must be at least 18");
  }
  if (!student.email.includes("@")) {
    throw new Error("Invalid email");
  }
  return {
    id: student.id,
    name: student.name.trim(),
    age: student.age,
```

```
    email: student.email.toLowerCase(),
    status: "Registered"
  };
}
```

Mandatory tasks (complete all four):

1. Create at least 12 Jest tests covering valid registration; ages 17, 18, and 120; blank names; malformed emails; missing properties; and incorrect input types.
2. Refactor validation so ID is a positive safe integer, name is a nonblank string, age is an integer from 18 to 120, and email matches a documented basic format; assert errors for invalid inputs.
3. Verify the input object is unchanged after successful registration; include one meaningful comparison of toEqual() and toStrictEqual() using nested data.
4. Run the tests before and after correction; submit the test file, corrected module, and a defect log listing input, observed defect, and resolution.

## Question 2: Student Gradebook and Stateful Classes (5 marks)

**Scenario and requirements.** An instructor maintains a gradebook for each student. Each GradeBook instance must manage its own scores, calculate meaningful statistics, and reject invalid marks. The starter implementation mishandles an empty gradebook and accepts unvalidated scores. Extend the class and prove that tests remain independent.

Starter file: src/grading.js

```
export class GradeBook {
  constructor(studentId) {
    this.studentId = studentId;
    this.scores = [];
  }
  addScore(score) { this.scores.push(score); }
  getAverage() {
    if (this.scores.length === 0) return 0;
    const total = this.scores.reduce((sum, score) => sum + score, 0);
    return total / this.scores.length;
  }
  getHighestScore() { return Math.max(...this.scores); }
  static isPassing(average) { return average >= 50; }
}
```

Mandatory tasks (complete all four):

1. Test empty, single, and multiple scores; duplicate scores; 0 and 100; and independent GradeBook instances.
2. Validate that every score is a finite number from 0 to 100; define getHighestScore() to return null for an empty gradebook and test this rule.
3. Implement removeLowestScore() to remove one occurrence of the lowest score and return the removed value, or null when empty; test ties and single-score cases.
4. Test isPassing() at 49.99, 50, and 50.01; use fresh instances to demonstrate no state leakage between tests.

## Question 3: Tuition Fee Calculation and Array Processing (5 marks)

**Scenario and requirements.** The finance office calculates tuition from enrolled courses and applies a scholarship discount. Invalid course details and floating-point arithmetic can produce incorrect charges. Build tests for fee calculations, define a precise monetary policy, and ensure the original course records remain unchanged.

Starter file: src/tuition.js

```
export function calculateTuition(courses, scholarshipRate = 0) {
  const subtotal = courses.reduce(
    (sum, course) => sum + course.creditHours * course.pricePerCredit, 0
```

```
  );
  const discount = subtotal * scholarshipRate;
  return { subtotal, discount, payable: subtotal - discount };
}
```

Mandatory tasks (complete all four):

1. Test no courses, one course, several courses, and omitted/undefined scholarshipRate; explicitly reject null as an invalid rate.
2. Enforce nonnegative integer creditHours, nonnegative finite pricePerCredit, and scholarshipRate between 0 and 1; assert TypeError for wrong types and RangeError for invalid numeric ranges.
3. Round subtotal, discount, and payable to two decimal places using a stated rounding policy; assert that no input course object is mutated.
4. Implement findMostExpensiveCourse() based on creditHours × pricePerCredit; return null for an empty array and the first matching course in a tie, then test both rules.

## Question 4: Enrollment Management and Side Effects (5 marks)

**Scenario and requirements.** When a student enrolls in a course, the system stores an enrollment and sends a notification. Duplicate enrollments and notification failures can leave inconsistent records. Use Jest mocks to verify both state changes and interactions, and implement an explicit failure-handling policy.

Starter file: src/enrollment.js

```
export class EnrollmentManager {
  constructor(notifier) {
    this.enrollments = [];
    this.notifier = notifier;
  }
  enroll(studentId, courseId) {
    const record = { studentId, courseId };
    this.enrollments.push(record);
    this.notifier.send(studentId, `Enrolled in course ${courseId}`);
    return record;
  }
  getEnrollments(studentId) {
    return this.enrollments.filter(r => r.studentId === studentId);
  }
}
```

Mandatory tasks (complete all four):

1. Mock notifier.send with jest.fn(); verify enrollment record, notification text, call arguments, and call count.
2. Reject duplicate studentId/courseId pairs without adding a record or sending another notification; test two students sharing a course.
3. If notification throws, roll back the newly added enrollment and rethrow the error; verify both outcomes with Jest.
4. Implement unenroll() to return true when a matching enrollment is removed and false otherwise; test isolation and return defensive copies from getEnrollments().

## Question 5: Test Isolation in a Student Repository (6 marks)

**Scenario and requirements.** A repository test suite passes when tests run alone but fails when run together because records are shared between tests. Reproduce the defect, then restructure the suite so every test starts from a known state. Include reusable fixtures and verify duplicate-ID handling.

Starter file: src/studentRepository.js

```
export class StudentRepository {
  constructor() { this.students = []; }
```

```
  add(student) { this.students.push(student); return student; }
  findById(id) { return this.students.find(s => s.id === id); }
  remove(id) { this.students = this.students.filter(s => s.id !== id); }
  count() { return this.students.length; }
}
```

Mandatory tasks (complete all four):

1. Reproduce a test-order failure using one shared StudentRepository and capture the failure evidence.
2. Replace shared state with beforeEach(), nested describe() groups, and a createStudentFixture(overrides) factory.
3. Reject duplicate student IDs and verify the error; use afterEach() to restore any spies or mocks.
4. Run a single test, the repository file, and the full suite; provide commands and results showing independent execution.

## Question 6: Lifecycle Hooks and Resource Cleanup (7 marks)

**Scenario and requirements.** A simulated database must be connected before records can be inserted and disconnected after the suite finishes. The team needs confidence that lifecycle hooks execute in the expected order and that no test inherits stale records. Demonstrate setup, isolation, and cleanup through executable Jest tests.

Starter file: src/fakeDatabase.js

```
export class FakeDatabase {
  constructor() { this.connected = false; this.records = []; }
  connect() { this.connected = true; }
  insert(record) {
    if (!this.connected) throw new Error("Database not connected");
    this.records.push(record);
  }
  disconnect() { this.connected = false; }
}
```

Mandatory tasks (complete all four):

1. Use beforeAll() to connect, beforeEach() to reset records, and afterAll() to disconnect the FakeDatabase.
2. Test successful insert when connected, rejection when disconnected, and no record leakage between test cases.
3. Add a nested describe() with scoped hooks and an event log; assert the observed hook order.
4. Verify cleanup using a disconnect() spy and an intentionally rejected operation caught with toThrow(); do not leave a failing test in the submitted suite.

## Question 7: Scholarship Eligibility Decision Table (7 marks)

**Scenario and requirements.** The scholarship committee awards full or partial scholarships according to academic average, attendance, completed credits, and disciplinary status. A single missed boundary or incorrect combination could award the wrong scholarship. Use decision-table thinking and data-driven Jest tests to verify every outcome.

Starter file: src/scholarship.js

```
export function evaluateScholarship(student) {
  const { average, attendance, credits, disciplinaryAction } = student;
  if (disciplinaryAction) return "Rejected";
  if (average >= 90 && attendance >= 95 && credits >= 15)
    return "Full Scholarship";
  if (average >= 80 && attendance >= 85 && credits >= 12)
    return "Partial Scholarship";
  return "Not Eligible";
}
```

Mandatory tasks (complete all four):

1. Build at least 20 object-based test.each() cases covering all four scholarship outcomes and the thresholds 80/90, 85/95, and 12/15.
2. Include single-condition failures and disciplinaryAction=true for otherwise full- and partial-eligible students.
3. Validate average and attendance as finite numbers from 0 to 100, credits as a nonnegative integer, and disciplinaryAction as Boolean; test invalid inputs.
4. Store cases in tests/fixtures/scholarshipCases.js and demonstrate a small equivalent array-based test table.

## Question 8: Tuition Payment Rules and Data Combinations (7 marks)

**Scenario and requirements.** The tuition payment service supports bank and card payments, with an additional fee for international transactions. Different methods and invalid amounts create a large combination space. Build a parameterized suite that checks calculated totals, rejects unsupported inputs, and documents the rounding rule.

Starter file: src/payment.js

```
export function processPayment(amount, method, isInternational) {
  if (amount <= 0) throw new Error("Invalid amount");
  let fee = 0;
  if (method === "card") fee = amount * 0.025;
  else if (method === "bank") fee = 10;
  else throw new Error("Unsupported payment method");
  if (isInternational) fee += amount * 0.015;
  return amount + fee;
}
```

Mandatory tasks (complete all four):

1. Use describe.each() for card/bank and test.each() for domestic/international combinations with explicit expected totals.
2. Create at least 16 cases including amount 0, negative amounts, decimals, large amounts, missing arguments, and unsupported methods.
3. Require positive finite amount, method card or bank, and Boolean isInternational; round the final payment to two decimal places and test the rule.
4. Submit a test-case matrix identifying each essential combination, expected value or exception, and actual Jest result.

## Question 9: Data-Driven Student Result Processing (6 marks)

**Scenario and requirements.** The academic office calculates a pass/fail result from an array of assessment scores. Invalid score types can silently corrupt the average. Create a reusable test dataset, define the valid score contract, and verify both result correctness and protection of the original input.

Starter file: src/resultProcessor.js

```
export function processResult(scores) {
  if (scores.length === 0) return { average: 0, status: "No Scores" };
  const average = scores.reduce((sum, score) => sum + score, 0)
    / scores.length;
  return { average, status: average >= 50 ? "Pass" : "Fail" };
}
```

Mandatory tasks (complete all four):

1. Create at least 15 test.each() cases covering empty arrays, 0/100, duplicates, decimals, and averages immediately below and at 50.
2. Test strings, nested arrays, NaN, Infinity, and out-of-range scores as invalid inputs.

3. Refactor to accept only arrays of finite numeric scores from 0 to 100; assert TypeError/RangeError and verify the array is unchanged.
4. Submit the parameterized test file and a result table containing case ID, input, expected result, actual result, and status.

## Question 10: Admission System Coverage Improvement (7 marks)

**Scenario and requirements.** An admission decision function contains several early-return rules for age, score, documentation, fees, international review, and scholarships. Two successful tests do not exercise most decisions. Use Jest coverage reports to identify gaps and design targeted tests for reachable outcomes and boundaries.

Starter file: src/admission.js

```
export function evaluateAdmission(applicant) {
  const { age, score, hasCertificate, applicationFeePaid,
    isInternational } = applicant;
  if (age < 18) return { status: "Rejected", reason: "Underage" };
  if (score < 60) return { status: "Rejected", reason: "Low Score" };
  if (!hasCertificate)
    return { status: "Pending", reason: "Missing Certificate" };
  if (!applicationFeePaid)
    return { status: "Pending", reason: "Payment Required" };
  if (isInternational && score < 75)
    return { status: "Pending", reason: "International Review" };
  if (score >= 90)
    return { status: "Accepted", scholarship: true };
  return { status: "Accepted", scholarship: false };
}
```

Mandatory tasks (complete all four):

1. Write only two initial successful-admission tests; run npm run test:coverage and record the four baseline metrics for admission.js.
2. Inspect the HTML report; add tests for every return path and boundaries age 17/18 and score 59/60, 74/75, 89/90.
3. Target 100% statement and branch coverage for admission.js; investigate and document any uncovered reachable branch.
4. Provide before/after coverage evidence and explain one admission-rule combination not guaranteed by branch coverage alone.

| Metric | Initial | Final | Improvement |
|---|---|---|---|
| **Statements** | Measured | Measured | Calculate |
| **Branches** | Measured | Measured | Calculate |
| **Functions** | Measured | Measured | Calculate |
| **Lines** | Measured | Measured | Calculate |

## Question 11: Coverage Thresholds and Regression Testing (6 marks)

**Scenario and requirements.** Before release, the team requires minimum statement, branch, function, and line coverage. New code may cause a previously passing build to fail. Configure measurable thresholds, introduce a controlled regression in coverage, and restore the threshold by writing meaningful tests.

Starter file: jest.config.js

```
coverageThreshold: {
  global: {
    statements: 90,
    branches: 85,
```

```
      functions: 90,
      lines: 90
    }
  }
```

Mandatory tasks (complete all four):

1. Add the provided global thresholds (90% statements, 85% branches, 90% functions, 90% lines) to jest.config.js and record the initial result.
2. Introduce a small new exported function with at least two untested branches and record the change in coverage.
3. Add assertions for the new branches until the configured thresholds pass; compare global with per-file metrics.
4. Submit a before/after report and a passing example of high execution coverage with an intentionally weak assertion, explaining why coverage alone is insufficient.

## Question 12: First-Order Mutation Testing of Scholarship Rules (7 marks)

**Scenario and requirements.** The scholarship module already has passing Jest tests, but passing tests do not prove that the suite detects subtle implementation errors. Run StrykerJS, inspect first-order mutants, and create targeted tests that kill meaningful survivors. Compare actual mutation results before and after improvement.

Starter file: src/scholarship.js

Mandatory tasks (complete all four):

1. Run the existing scholarship tests, then StrykerJS targeting src/scholarship.js; save counts for killed, survived, timeout, and no-coverage mutants.
2. Select at least five first-order mutants from the report and describe the changed condition or returned value.
3. Add at least five targeted tests intended to kill surviving non-equivalent mutants; document each test input and expected outcome.
4. Rerun mutation testing; submit initial/final scores and a five-row mutant analysis table with observed statuses.

| Mutant ID | Mutation | Initial status | New test | Final status |
|---|---|---|---|---|
| M1 | From report | Record | Design | Record |
| M2 | From report | Record | Design | Record |
| M3 | From report | Record | Design | Record |
| M4 | From report | Record | Design | Record |
| M5 | From report | Record | Design | Record |

## Question 13: First-Order and Higher-Order Mutant Investigation (6 marks)

**Scenario and requirements.** A simple eligibility decision combines three conditions. Changing one operator creates a first-order mutant; combining changes creates higher-order mutants whose behavior may interact. Implement the specified mutants separately and design valid inputs that distinguish each mutant from the original.

Starter file: src/eligibility.js

```
export function checkEligibility(age, score, hasDocument) {
  if (age >= 18 && score >= 60 && hasDocument) return "Eligible";
  return "Not Eligible";
}
```

Mandatory tasks (complete all four):

1. Write baseline tests for checkEligibility() using integer ages 0–120, integer scores 0–100, and Boolean hasDocument; create three independent FOMs (age >= to >, score >= to >, first && to ||).

2. Create one second-order mutant combining the two relational changes and one third-order mutant combining all three changes.
3. Use test.each() to compare original and each mutant on valid inputs; a mutant is killed when its output differs from the original.
4. Submit a matrix of mutant, killing input, original output, mutant output, and a short explanation of interactions.

## Question 14: Mutation Score Improvement Project (7 marks)

**Scenario and requirements.** The payment module is scheduled for release. The quality team must assess how sensitive its Jest tests are to arithmetic and conditional defects. Use StrykerJS to identify survivors, improve the suite, and justify any mutants that cannot be distinguished under the specified business rules.

Starter file: src/payment.js

Mandatory tasks (complete all four):

1. Configure StrykerJS to mutate src/payment.js and save the initial report after confirming Jest tests pass.
2. Classify surviving mutants affecting fee arithmetic, international fee logic, and payment-method errors.
3. Add at least eight targeted Jest tests, rerun mutation testing, and record changed mutant statuses.
4. Calculate the score using report counts, compare it with StrykerJS, and justify any potentially equivalent mutant.
1. Manual scoring convention: Mutation Score = (Killed + Timeout) / (Killed + Timeout + Survived + No Coverage) × 100%. Equivalent mutants require separate justification and are not automatically counted as killed.